

```

# Install required libraries
!pip install opencv-python matplotlib

import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# -----
# Upload Image
# -----
uploaded = files.upload()
image_path = list(uploaded.keys())[0]

# Read image
img = cv2.imread(image_path)
img_color = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# -----
# Define rotation angles
# -----
angles = [0, 30, 60, 90, 120, 150, 180]

# ORB detector
orb = cv2.ORB_create(nfeatures=1000)

# Detect features in original image
kp1, des1 = orb.detectAndCompute(gray, None)

results = []
accuracies = []

# -----
# Rotate & Match
# -----
for angle in angles:
    h, w = gray.shape
    center = (w//2, h//2)

    # Rotation matrix
    M = cv2.getRotationMatrix2D(center, angle, 1.0)
    rotated = cv2.warpAffine(gray, M, (w, h))

    # Detect features
    kp2, des2 = orb.detectAndCompute(rotated, None)

    # Match features
    bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

    if des2 is not None and len(des2) > 0:
        matches = bf.match(des1, des2)
        matches = sorted(matches, key=lambda x: x.distance)

        good_matches = matches[:int(len(matches)*0.3)] # top 30%
        accuracy = (len(good_matches) / len(matches)) * 100 if len(matches) > 0 else 0
    else:
        matches = []
        good_matches = []
        accuracy = 0

    accuracies.append(accuracy)

# Draw matches
matched_img = cv2.drawMatches(img_color, kp1,
                              cv2.cvtColor(rotated, cv2.COLOR_GRAY2RGB), kp2,
                              good_matches, None, flags=2)

results.append(matched_img)

# -----
# Display Results
# -----
plt.figure(figsize=(15,10))

for i, angle in enumerate(angles):
    plt.subplot(3,3,i+1)
    plt.imshow(results[i])
    plt.title(f"{angle}°\nAccuracy: {accuracies[i]:.2f}%")
    plt.axis('off')

```

```
plt.tight_layout()
plt.show()

# -----
# Plot Accuracy vs Angle
# -----
plt.figure()
plt.plot(angles, accuracies, marker='o')
plt.xlabel("Rotation Angle (degrees)")
plt.ylabel("Matching Accuracy (%)")
plt.title("Rotation Impact on Feature Matching")
plt.grid()
plt.show()

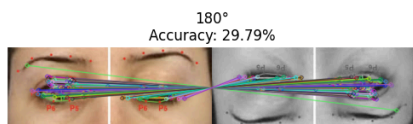
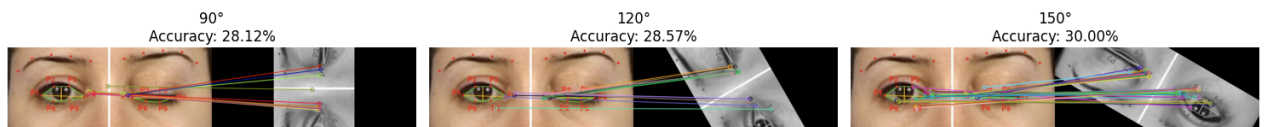
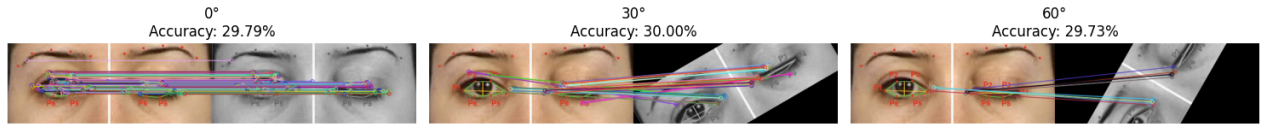
# -----
# Print Results Table
# -----
print("----- MATCHING ACCURACY -----")
for i in range(len(angles)):
    print(f"Angle {angles[i]}° -> Accuracy: {accuracies[i]:.2f}%")
```

Requirement already satisfied: opencv-python in /usr/local/lib/python3.12/dist-packages (5.0.0.93)
 Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
 Requirement already satisfied: numpy>=2 in /usr/local/lib/python3.12/dist-packages (from opencv-python) (2.0.2)
 Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
 Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
 Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.63.0)
 Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.5.0)
 Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (26.2)
 Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
 Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
 Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post0)
 Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)

ear.jpg

ear.jpg(image/jpeg) - 7216 bytes, last modified: 7/31/2026 - 100% done

Saving ear.jpg to ear.jpg



Rotation Impact on Feature Matching

